

DATE: 6/02/2026
EX NO :5

NAME: Faatina S
REG NO :3122 23 5001 046

UCS2612 & Machine Learning Laboratory

Experiment 5: Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning

Aim

To implement and compare **Perceptron Learning Algorithm (PLA)** and **Multi-Layer Perceptron (MLP)** for handwritten character classification, and to analyze the impact of hyperparameter tuning on model performance, convergence, and generalization.

Objective

- To preprocess handwritten character images using cropping, resizing, normalization, and feature extraction techniques.
- To implement the Perceptron Learning Algorithm (PLA) for character classification.
- To design and train a Multi-Layer Perceptron (MLP) with hidden layers for multi-class recognition.
- To perform hyperparameter tuning by varying activation functions, optimizers, learning rates, and batch sizes.
- To evaluate and compare PLA and MLP using performance metrics such as accuracy, precision, recall, F1-score, confusion matrix, and ROC curves.

Implementation

PLA FROM STRATCH

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

#Loading dataset
df=pd.read_csv("/Users/faatinashahul/Desktop/ml/hand/english.csv")
print(df.head(5))
print(df.columns)
```

```
      image label
0  Img/img001-001.png    0
1  Img/img001-002.png    0
2  Img/img001-003.png    0
3  Img/img001-004.png    0
4  Img/img001-005.png    0
Index(['image', 'label'], dtype='object')
```

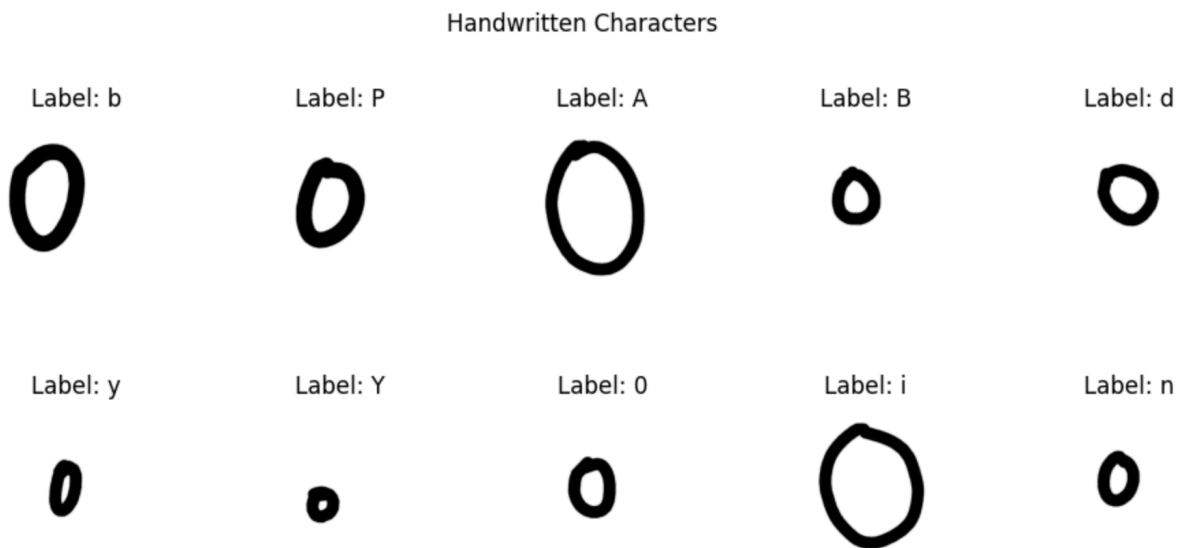
```

import cv2
import os
import random

path = "/Users/faatinashahul/Desktop/ml/hand/"

random_indices=random.sample(range(len(df)), 10)
plt.figure(figsize=(12,5))
for i,idx in enumerate(random_indices):
    img_path=os.path.join(path,df.iloc[i]['image'])
    label=df.iloc[idx]['label']
    img=cv2.imread(img_path,cv2.IMREAD_GRAYSCALE)
    plt.subplot(2,5,i+1)
    plt.imshow(img,cmap='gray')
    plt.title(f"Label: {label}")
    plt.axis('off')
plt.suptitle("Handwritten Characters")
plt.show()

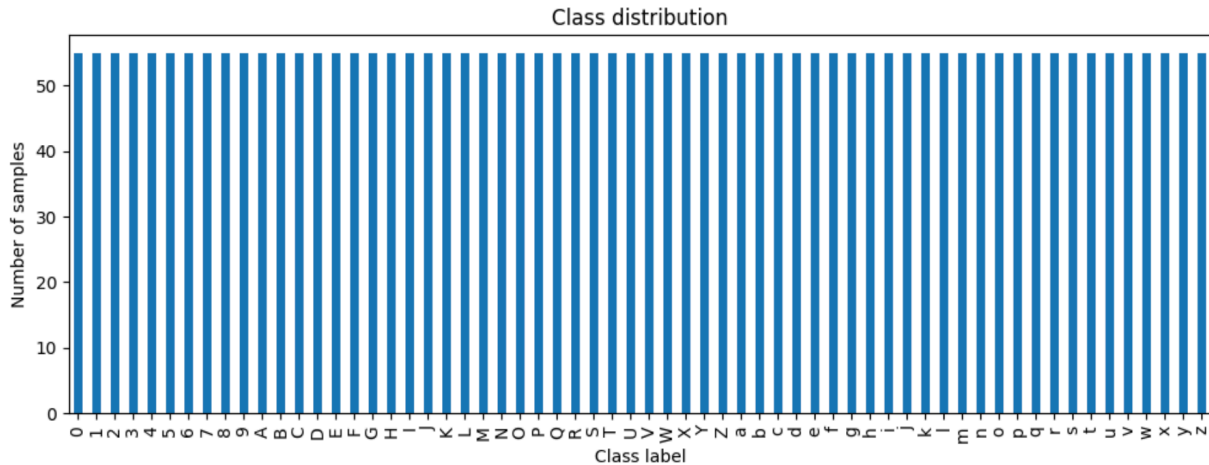
```



```

#Class distribution
plt.figure(figsize=(12,4))
df['label'].value_counts().sort_index().plot(kind='bar')
plt.title("Class distribution")
plt.xlabel("Class label")
plt.ylabel("Number of samples")
plt.show()

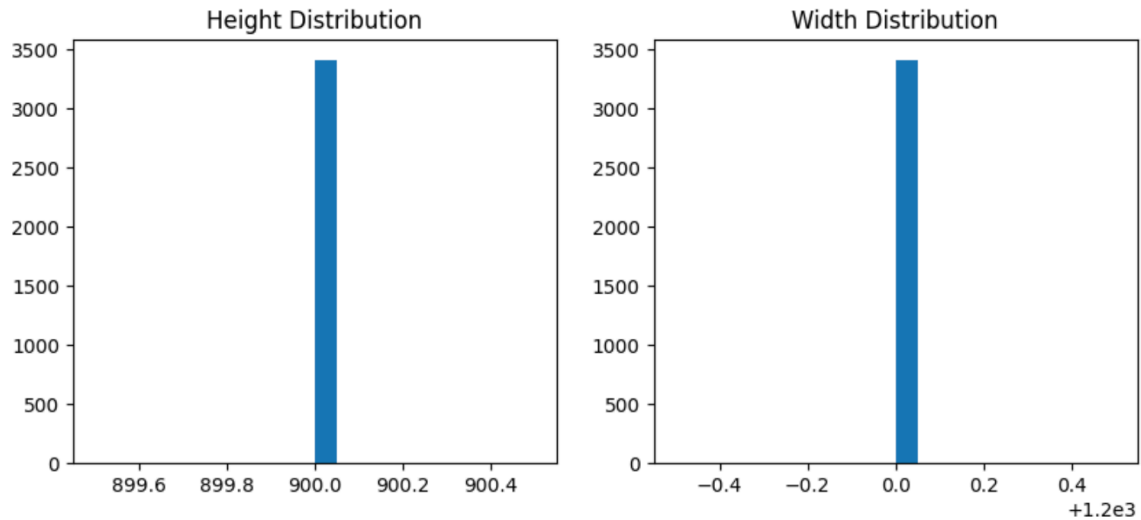
```



```
random_indices=random.sample(range(len(df)),20)
sizes=[]
for idx in random_indices:
    img_path=os.path.join(path,df.iloc[idx]['image'])
    img=cv2.imread(img_path,cv2.IMREAD_GRAYSCALE)
    if img is not None:
        sizes.append(img.shape)
print("Original image sizes(height,width):")
print(set(sizes))
```

```
Original image sizes(height,width):
{(900, 1200)}
```

```
IMG_SIZE=28
heights = []
widths = []
for _, row in df.iterrows():
    img_path = os.path.join(path, row['image'])
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    h, w = img.shape
    heights.append(h)
    widths.append(w)
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.hist(heights, bins=20)
plt.title("Height Distribution")
plt.subplot(1,2,2)
plt.hist(widths, bins=20)
plt.title("Width Distribution")
plt.show()
```



```
def preprocess_images(df):
    X=[]
    y=[]
    for i,row in df.iterrows():
        img_path=os.path.join(path,row['image'])
        img=cv2.imread(img_path,cv2.IMREAD_GRAYSCALE)
        if img is None:
            continue
        #cropping white background
        _,thresh=cv2.threshold(img,240,255,cv2.THRESH_BINARY_INV)
        contours,_=cv2.findContours(thresh,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)
        if contours:
            c = max(contours, key=cv2.contourArea)
            x, y_coord, w, h = cv2.boundingRect(c)
            img = img[y_coord:y_coord+h, x:x+w]
            img=cv2.resize(img,(IMG_SIZE,IMG_SIZE))
            #normalizing and flattening
            img=img/255.0
            img=img.flatten()
            X.append(img)
            y.append(row['label'])
    return np.array(X),np.array(y)
X,y=preprocess_images(df)
print("Feature matrix shape:",X.shape)
print("Label vector shape:",y.shape)
```

Feature matrix shape: (3410, 784)
 Label vector shape: (3410,)

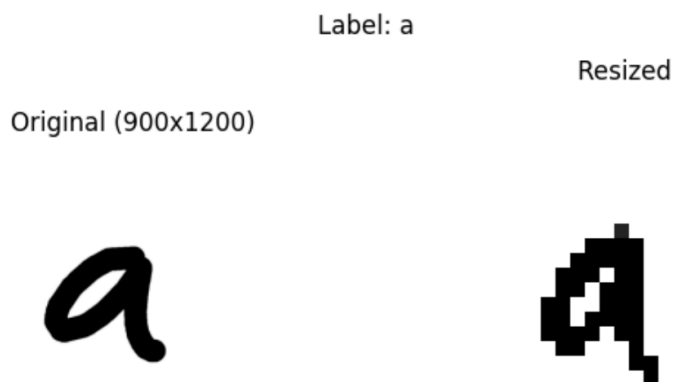
```

idx=np.random.randint(0, len(df))
img_path=os.path.join(path, df.iloc[idx]['image'])
img_original=cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
img_resized=cv2.resize(img_original, (IMG_SIZE, IMG_SIZE))

plt.figure(figsize=(8,4))
plt.subplot(1,2,1)
plt.imshow(img_original, cmap='gray')
plt.title("Original (900x1200)")
plt.axis('off')

plt.subplot(1,2,2)
plt.imshow(img_resized, cmap='gray')
plt.title("Resized ")
plt.axis('off')
plt.suptitle(f"Label: {df.iloc[idx]['label']}")
plt.show()

```



```

#Splitting the dataset
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(
    X,y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
print("Training set size: ",X_train.shape)
print("Testing set size: ",X_test.shape)

```

Training set size: (2728, 784)
 Testing set size: (682, 784)

```

#step activation function
def step_activation(x):
    return np.where(x>=0,1,0)
#perceptron class
class Perceptron:
    def __init__(self,lr=0.01,epochs=50):
        self.lr=lr
        self.epochs=epochs

    def fit(self,X,y):
        self.weights=np.zeros(X.shape[1])
        self.bias=0
        self.errors=[]

        for _ in range(self.epochs):
            total_error=0
            for xi,yi in zip(X,y):
                linear_output=np.dot(xi,self.weights)+self.bias
                y_hat=step_activation(linear_output)
                update=self.lr * (yi - y_hat)
                self.weights+=update*xi
                self.bias+=update
                total_error+=abs(update)
            self.errors.append(total_error)

    def score(self,X):
        return np.dot(X,self.weights)+self.bias

```

```

#One-vs-rest PLA for multi-class
classes=np.unique(y_train)
pla_models=[]
for cls in classes:
    y_binary=np.where(y_train==cls,1,0)
    pla=Perceptron(lr=0.01,epochs=50)
    pla.fit(X_train,y_binary)
    pla_models.append(pla)

def pla_predict(X, pla_models, classes):
    scores=[]
    for pla in pla_models:
        scores.append(pla.score(X))
    scores = np.array(scores)
    best_class_index = np.argmax(scores, axis=0)
    return classes[best_class_index]

def pla_scores(X, pla_models):
    scores = []

```

```

for pla in pla_models:
    scores.append(pla.score(X))
return np.array(scores).T

y_pred_pla=pla_predict(X_test,pla_models,classes)

from sklearn.metrics import accuracy_score,f1_score,precision_score,recall_score
#PLA metrics
print("PLA Accuracy: ",accuracy_score(y_test,y_pred_pla))
print("PLA Precision: ",precision_score(y_test,y_pred_pla,average='macro'))
print("PLA Recall: ",recall_score(y_test,y_pred_pla,average='macro'))
print("PLA F1-score: ",f1_score(y_test,y_pred_pla,average='macro'))

#plotting PLA training errors
plt.figure()
for i,pla in enumerate(pla_models[:5]):
    plt.plot(pla.errors,label=f'Class {classes[i]}')
plt.title("PLA Training Error vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Total error")
plt.legend()
plt.show()

```

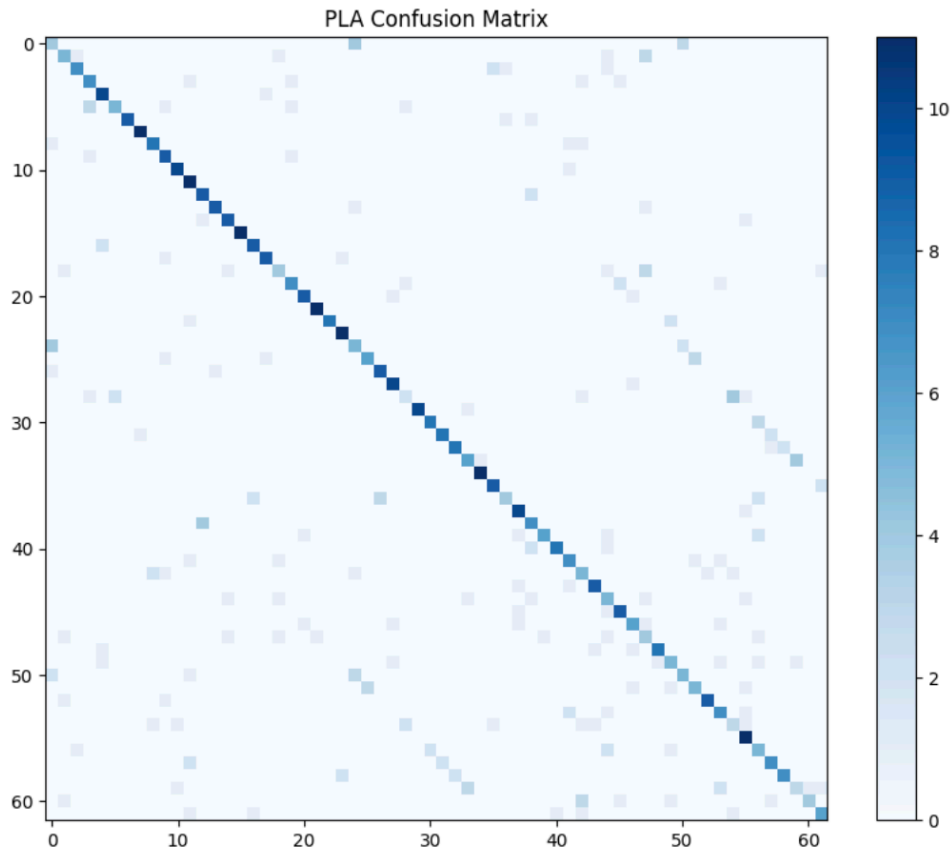
Training set size: (2728, 784)
 Testing set size: (682, 784)
 PLA Accuracy: 0.6730205278592375
 PLA Precision: 0.6812653744105357
 PLA Recall: 0.6730205278592375
 PLA F1-score: 0.6684131192601275



```

from sklearn.metrics import confusion_matrix
# Confusion Matrix
plt.figure(figsize=(10,8))
cm = confusion_matrix(y_test, y_pred_pla, labels=classes)
plt.imshow(cm, cmap='Blues')
plt.title("PLA Confusion Matrix")
plt.colorbar()
plt.show()

```



```

from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import roc_curve, auc
y_score_pla = pla_scores(X_test, pla_models)
lb = LabelBinarizer()
y_test_bin = lb.fit_transform(y_test)
fpr = dict()
tpr = dict()
roc_auc = dict()
# Per-class ROC
for i in range(len(classes)):
    fpr[i], tpr[i], _ = roc_curve(
        y_test_bin[:, i],
        y_score_pla[:, i]
    )

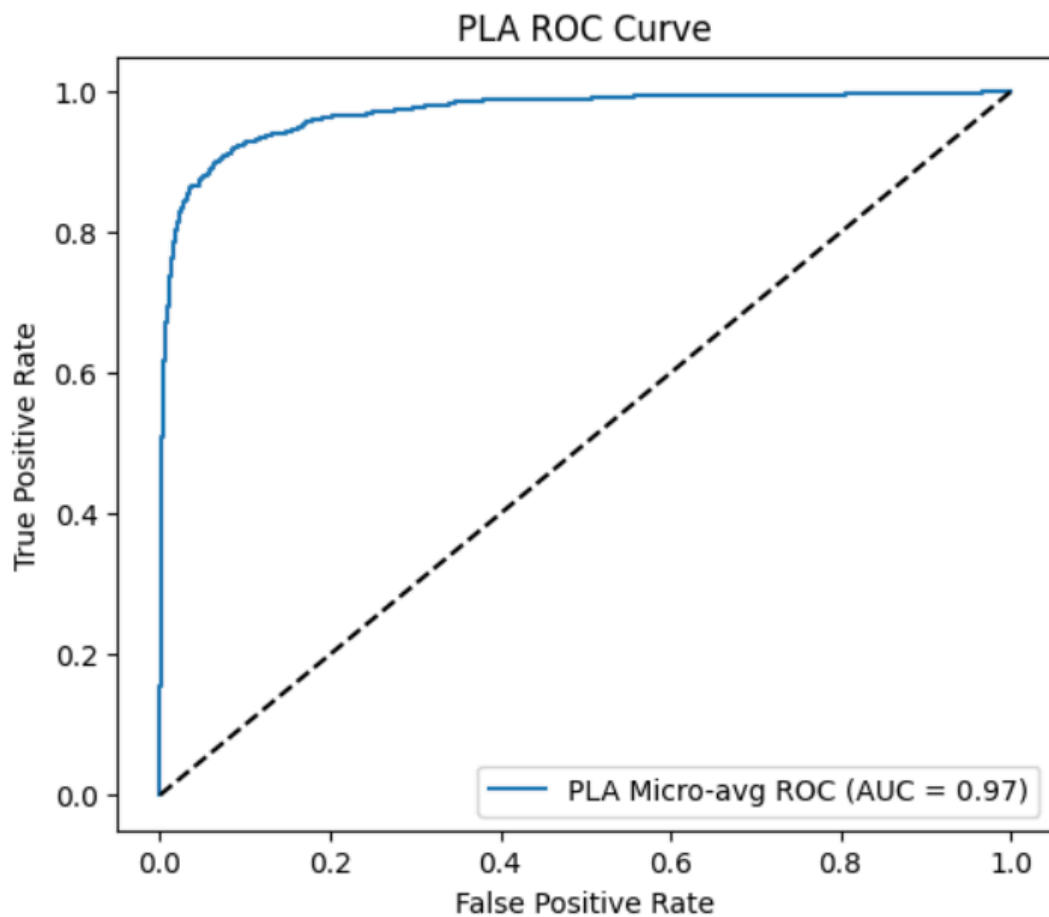
```

```

    )
    roc_auc[i] = auc(fpr[i], tpr[i])

# Micro-average ROC
fpr["micro"], tpr["micro"], _ = roc_curve(y_test_bin.ravel(), y_score_pla.ravel())
)
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
plt.figure(figsize=(6,5))
plt.plot(
    fpr["micro"],
    tpr["micro"],
    label=f'PLA Micro-avg ROC (AUC = {roc_auc["micro"]:.2f})'
)
plt.plot([0,1],[0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("PLA ROC Curve")
plt.legend()
plt.show()

```



MLP

```
from sklearn.neural_network import MLPClassifier

mlp_configs=[
{"hidden_layer_sizes":(256,128),"activation":"relu","solver":"adam","lr":0.001,"batch_size":32},
{"hidden_layer_sizes":(256,128),"activation":"relu","solver":"adam","lr":0.01,"batch_size":32},
{"hidden_layer_sizes":(256,128),"activation":"relu","solver":"adam","lr":0.001,"batch_size":64},
{"hidden_layer_sizes":(256,128),"activation":"relu","solver":"sgd","lr":0.001,"batch_size":32},
{"hidden_layer_sizes":(256,128),"activation":"tanh","solver":"adam","lr":0.001,"batch_size":32}
]

mlp_results=[]
for cfg in mlp_configs:

mlp=MLPClassifier(hidden_layer_sizes=cfg["hidden_layer_sizes"],activation=cfg["activation"],solver=cfg['solver'],learning_rate_init=cfg['lr'],
                  batch_size=cfg['batch_size'],max_iter=100,random_state=42,verbose=False)
mlp.fit(X_train,y_train)
y_pred_mlp=mlp.predict(X_test)

acc=accuracy_score(y_test, y_pred_mlp)
prec=precision_score(y_test, y_pred_mlp, average='macro')
rec=recall_score(y_test, y_pred_mlp, average='macro')
f1=f1_score(y_test, y_pred_mlp, average='macro')

mlp_results.append({
    "Hidden Layers": cfg["hidden_layer_sizes"],
    "Activation": cfg["activation"],
    "Solver": cfg["solver"],
    "Learning Rate": cfg["lr"],
    "Batch Size": cfg["batch_size"],
    "Accuracy": acc,
    "Precision": prec,
    "Recall": rec,
    "F1-score": f1,
    "Model": mlp
})

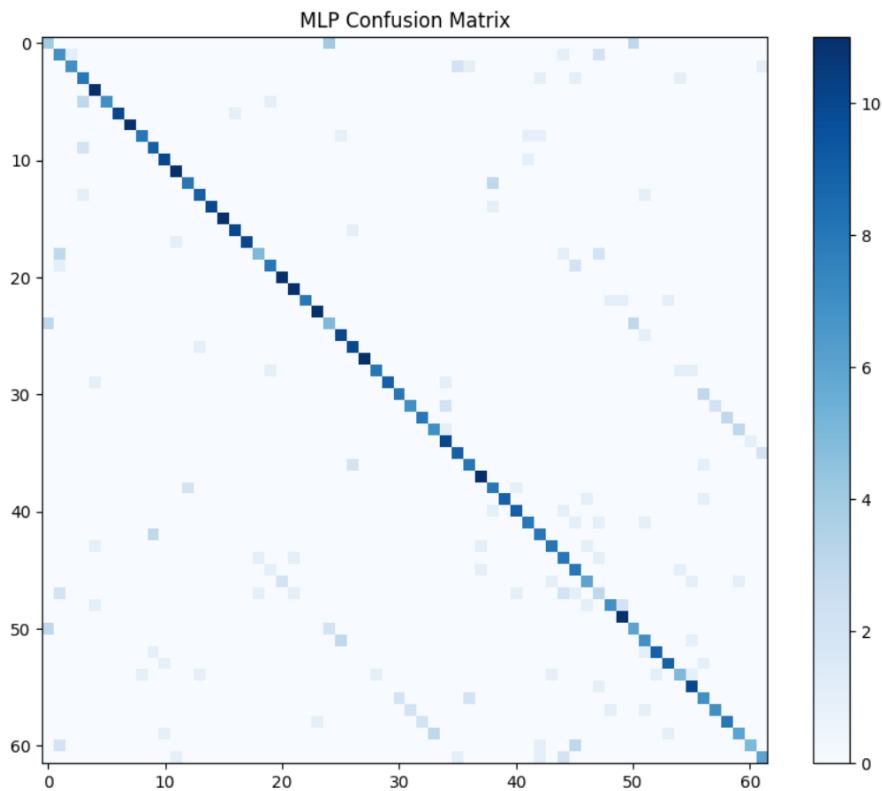
#converting results to dataframe
mlp_df=pd.DataFrame(mlp_results)
print("All MLP configurations and Performance")
print(mlp_df.drop(columns="Model").to_string(index=False))
```

Hidden Layers	Activation	Solver	Learning Rate	Batch Size	Accuracy	Precision	Recall	F1-score
(256, 128)	relu	adam	0.001	32	0.750733	0.758999	0.750733	0.749157
(256, 128)	relu	adam	0.010	32	0.607038	0.641625	0.607038	0.595331
(256, 128)	relu	adam	0.001	64	0.736070	0.741603	0.736070	0.732804
(256, 128)	relu	sgd	0.001	32	0.737537	0.755174	0.737537	0.735570
(256, 128)	tanh	adam	0.001	32	0.753666	0.764964	0.753666	0.752204

```
# best model based on accuracy
best_mlp = mlp_results[np.argmax([res["Accuracy"] for res in mlp_results])]
print("\nBest MLP Configuration")
best_df = pd.DataFrame([best_mlp]).drop(columns="Model")
print(best_df.to_string(index=False))
```

```
Best MLP Configuration
Hidden Layers Activation Solver Learning Rate Batch Size Accuracy Precision Recall F1-score
(256, 128)      tanh      adam          0.001          32  0.753666  0.764964  0.753666  0.752204
```

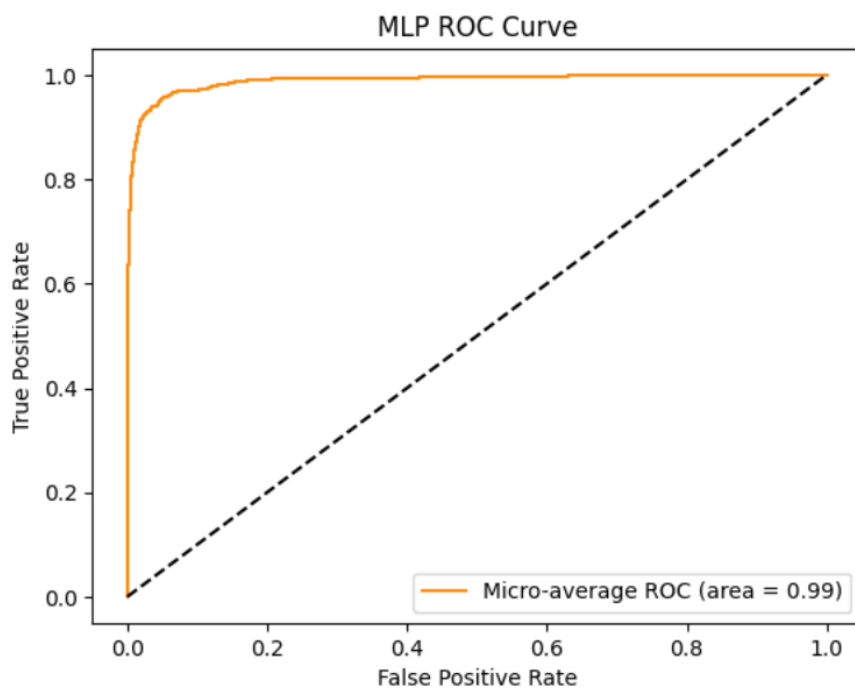
```
# Confusion Matrix
plt.figure(figsize=(10,8))
cm = confusion_matrix(y_test,best_mlp["Model"].predict(X_test),labels=classes
)
plt.imshow(cm, cmap='Blues')
plt.title("MLP Confusion Matrix")
plt.colorbar()
plt.show()
```



```

from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import LabelBinarizer
#ROC curve for multi-class
y_test_bin = LabelBinarizer().fit_transform(y_test)
y_score = best_mlp["Model"].predict_proba(X_test)
fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(len(classes)):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
# Micro-average
fpr["micro"], tpr["micro"], _ = roc_curve(
    y_test_bin.ravel(),
    y_score.ravel()
)
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
plt.figure()
plt.plot(fpr["micro"], tpr["micro"], label=f'Micro-average ROC (area = {roc_auc["micro"]:.2f})', color='darkorange')
)
plt.plot([0,1], [0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("MLP ROC Curve")
plt.legend()
plt.show()

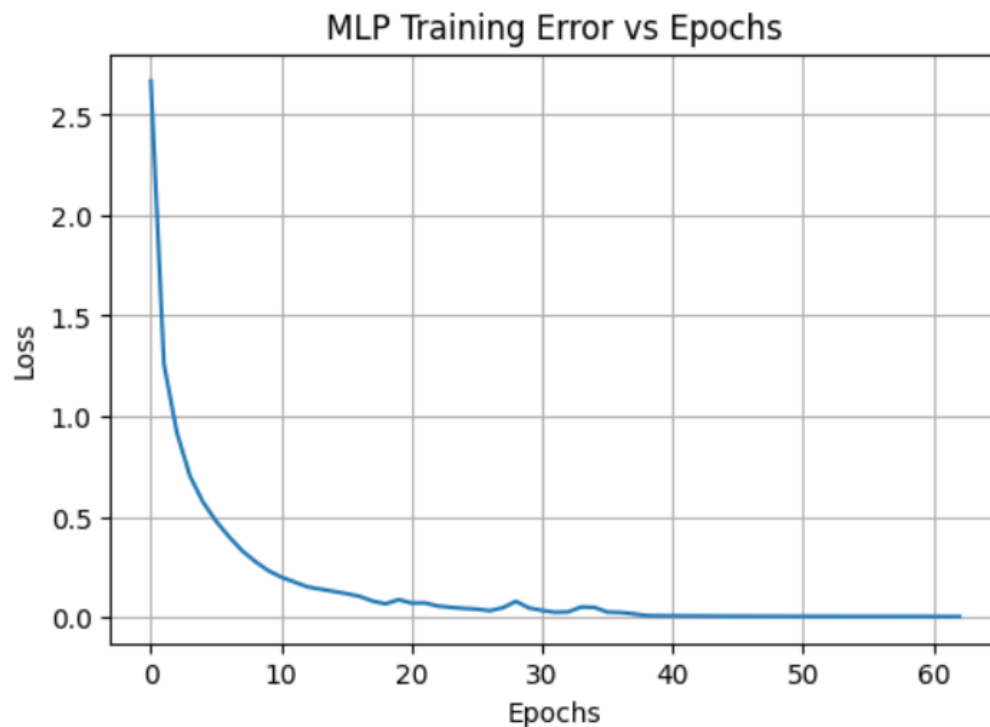
```



```

best_model = best_mlp["Model"]
plt.figure(figsize=(6,4))
plt.plot(best_model.loss_curve_)
plt.title("MLP Training Error vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.grid(True)
plt.show()

```



```

comparison = pd.DataFrame([
    ["PLA",accuracy_score(y_test, y_pred_pla),precision_score(y_test, y_pred_pla,
    average='macro'),recall_score(y_test, y_pred_pla, average='macro'),f1_score(y_test,
    y_pred_pla, average='macro')],
    ["MLP",best_mlp["Accuracy"],best_mlp["Precision"],best_mlp["Recall"],best_mlp["F1-
    score"]]],
    columns=["Model","Accuracy","Precision","Recall","F1-score"])
print("\nModel Comparison")
print(comparison)

```

Model Comparison					
	Model	Accuracy	Precision	Recall	F1-score
0	PLA	0.673021	0.681265	0.673021	0.668413
1	MLP	0.753666	0.764964	0.753666	0.752204

A/B EXPERIMENT COMPARISON (PLA vs MLP)

Justification for Chosen Hyperparameters

Hyperparameter tuning was performed by varying activation function, optimizer, learning rate, and batch size for the MLP model. Multiple configurations were evaluated and compared using accuracy, precision, recall, and F1-score. Among all tested models, the configuration with **hidden layers (256, 128), Tanh activation, Adam optimizer, learning rate 0.001, and batch size 32** produced the best performance. This model achieved the highest accuracy (75.36%) and F1-score (0.752).

Tanh activation performed slightly better than ReLU for this dataset as it captured subtle stroke variations in handwritten characters. Adam optimizer ensured faster and smoother convergence compared to SGD. A lower learning rate (0.001) provided stable weight updates, while batch size 32 improved generalization through more frequent gradient updates. Thus, this configuration was selected as the optimal model.

Metric	PLA	MLP
Accuracy	Lower	Higher
Precision	Lower	Higher
Recall	Lower	Higher
F1-Score	Lower	Higher
Convergence	Slow	Faster
Generalization	Poor	Good

Final Chosen Hyperparameters for MLP

Hyperparameter	Selected Value
Hidden Layers	(256, 128)

Activation	Tanh
Optimizer	Adam
Learning Rate	0.001
Batch Size	32

Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score
PLA	0.6730	0.6813	0.6730	0.6684
MLP	0.7537	0.7650	0.7537	0.7522

MLP outperformed PLA across all evaluation metrics.

Strengths and Weaknesses

Aspect	PLA	MLP
Decision Boundary	Linear	Non-linear
Learning Capability	Limited	High
Accuracy	Moderate	High
Training Speed	Fast	Slower
Feature Extraction	Cannot learn complex patterns	Learns hierarchical features
Multi-class Handling	Less effective	Highly effective

Impact of Hyperparameter Tuning

Hyperparameter tuning had a significant impact on model convergence and performance:

- Lower learning rate improved stability.
- Adam optimizer accelerated convergence compared to SGD.
- Tanh activation enhanced classification accuracy over ReLU.
- Smaller batch size improved generalization.

Tuning resulted in improved loss convergence, reduced training error, and higher classification accuracy for the MLP model.

Observation

1. Why does PLA underperform compared to MLP?

PLA is a linear classifier that can only learn linearly separable decision boundaries. Handwritten character data is highly complex and non-linear due to variations in strokes, shapes, and writing styles. MLP uses hidden layers with non-linear activation functions, enabling it to learn complex feature representations. Hence, MLP achieves higher accuracy than PLA.

2. Which hyperparameters had the most impact on MLP performance?

The most influential hyperparameters were:

- Activation function
- Optimizer
- Learning rate

Tanh activation produced slightly better accuracy than ReLU. Adam optimizer improved convergence speed and stability. A lower learning rate (0.001) resulted in smoother training and higher performance.

3. Did optimizer choice (SGD vs Adam) affect convergence?

Yes. Adam optimizer showed faster and more stable convergence compared to SGD. SGD required more iterations and exhibited slower loss reduction, while Adam adapted learning rates automatically, leading to improved accuracy.

4. Did adding more hidden layers always improve results? Why or why not?

No. Increasing hidden layers increases model capacity but does not always improve performance. Excessive layers may cause overfitting, increased computation time, and training instability. Optimal depth provides a balance between learning capability and generalization.

5. Did MLP show overfitting? How could it be mitigated?

Slight overfitting may occur when training accuracy exceeds testing accuracy. It can be mitigated using:

- Dropout regularization
- Early stopping
- L2 regularization
- Data augmentation
- Reducing model complexity

These techniques improve generalization.

Learning Outcomes

- Implemented and analyzed the Perceptron Learning Algorithm for linear classification.
- Designed and trained a Multi-Layer Perceptron for multi-class image classification.
- Performed image preprocessing including cropping, resizing, normalization, and feature extraction.
- Evaluated model performance using accuracy, precision, recall, F1-score, ROC curves, and confusion matrices.
- Compared linear and non-linear models to understand the impact of hyperparameter tuning on convergence and accuracy.